# ***PROGRAMACIÓN III: TRABAJO PRACTICO OBLIGATORIO***




**Universidad:** Universidad Argentina de la Empresa

**Carrera:** Licenciatura en Gestión de Tecnología de la Información

**Materia:** Programación III

**Turno:** Viernes Tarde

**Nombre y apellido de la profesora:**

- Leon, Maria Angela

**Nombre, apellido y Legajo de los integrantes del equipo:**

- Francisco Ferrero LU: 1208322
- Josue Palacios LU: 1207683

**Fecha de la primera entrega:** 23/10/2026

**Fecha de la segunda entrega:** 06/11/2026

# Informe Propuesta 2 – TPO Programación III

## 1. Funcionalidad 1: organización y consulta de pedidos

### 1.1 Análisis del problema

Cada pedido tiene un identificador, un centro de destino, una fecha límite, una prioridad y un peso total. El sistema debe ordenar los pedidos por fecha límite más cercana, ordenarlos por prioridad descendente y buscar un pedido por su identificador.

La búsqueda por identificador tiene que ser rápida aunque haya muchos pedidos, y para eso se usa búsqueda binaria, que solo funciona sobre datos ordenados. Por eso la lista de pedidos se mantiene ordenada por id en todo momento, insertando cada pedido nuevo en su posición en lugar de agregarlo al final. El costo temporal de agregar es de O(N) (siguiendo la estrategia de correr los elementos para hacer lugar al nuevo) y en cada búsqueda O(log N). Esta solución es superior a la opción de agregar al final del arreglo y ordenar antes de cada búsqueda O(N log N) por búsqueda o buscar recorriendo toda la lista O(N) por búsqueda; en ambas cada consulta por id sale más cara.

Reglas de desempate:

- **Por prioridad** (definida por la consigna): ante igual prioridad gana la fecha límite más cercana y, si también coincide, el identificador menor.
- **Por fecha** (por consecuencia del criterio anterior): ante igual fecha gana el identificador menor.

OBS: Los identificadores son únicos. Se lanza una excepción si se quiere insertar un pedido con un id ya existente.

### 1.2 Organización de los datos que se necesita en cada función principal

- **Buscar por id:** necesita una lista ordenada por id, porque la búsqueda binaria solo funciona con datos ordenados. GestorPedidos mantiene una única lista, pedidosPorId, siempre ordenada por id. Costo: O(log N).
- **Ordenar por fecha o por prioridad:** no necesita ninguna organización previa, porque recibe los datos y los ordena. Se ordena una copia de la lista y la lista interna no se modifica. Costo: O(N log N).
- **Agregar un pedido:** tiene que conservar el orden por id y detectar ids repetidos. Se busca la posición donde va el pedido y se lo inserta ahí, sin reordenar toda la lista. Costo: O(N).

La lista interna debe quedar siempre ordenada por id. Por eso los ordenamientos por fecha y por prioridad se calculan sobre una copia: si se ordenara la lista interna, dejaría de estar ordenada por id y la búsqueda binaria dejaría de funcionar.

### 1.3 Estrategia elegida: Divide y Conquista

**Ordenamiento: Merge Sort.** Divide la lista en dos mitades, ordena cada mitad recursivamente y combina las dos mitades ya ordenadas intercalándolas con merge. El caso

base es una sublista de un solo elemento. Se eligió porque garantiza O(N log N) sin importar el orden en que lleguen los datos.

**Búsqueda: búsqueda binaria.** Compara el id buscado con el del elemento del medio de la lista ordenada y descarta la mitad donde no puede estar; repite sobre la mitad que queda. El caso base es un solo elemento. Reduce el problema a la mitad en cada paso.

La estrategia se reconoce en el código: MergeSort.mergeSort hace dos llamadas recursivas sobre las mitades y luego llama a merge, y BusquedaBinaria.buscar se llama a sí misma sobre una de las mitades. ComparadorPedido comprende las reglas de orden (fecha, prioridad e id), de modo que el mismo Merge Sort sirve para ambos ordenamientos.

## 1.4 Alternativas descartadas

- **Selección e inserción:** son $O(N^2)$ y se vuelven impracticables al crecer la cantidad de pedidos.
- **Quick Sort:** es O(N log N) en promedio pero $O(N^2)$ en el peor caso.
- **Búsqueda lineal:** es O(N) y no aprovecha que la lista está ordenada por id.
- **Reordenar toda la lista en cada alta:** costaría O(N log N) por pedido agregado, contra O(N) de insertarlo en su posición.

## 1.5 Complejidad temporal

N es la cantidad de pedidos. Merge Sort y la búsqueda binaria son algoritmos recursivos de Divide y Conquista, así que su tiempo se obtiene con el teorema maestro, que usa tres valores:

- **a:** la cantidad de subproblemas en que se divide el problema.
- **b:** el factor en que se reduce el tamaño en cada subproblema (cada uno tiene tamaño N/b).
- **k:** el grado del polinomio que describe el costo del trabajo que se hace fuera de las llamadas recursivas, que es $O(N^k)$.

El resultado depende de cómo se compara a con $b^k$:

$$T(N) = \begin{cases} O(N^k) & \text{si } a < b^k \\ O(N^k \log N) & \text{si } a = b^k \\ O\left(N^{\log_b a}\right) & \text{si } a > b^k \end{cases}$$

**Merge Sort.** Divide la lista en 2 mitades (a = 2) de tamaño N/2 (b = 2). El trabajo fuera de las llamadas recursivas es el de merge, que recorre las dos mitades una vez y cuesta O(N); calcular el punto medio es O(1). Entonces ese trabajo es O(N) (k = 1).

Como $b^k = 2^1 = 2$ es igual a a, se aplica el caso $a = b^k$:

$$a = 2, \quad b^k = 2^1 = 2 \;\Rightarrow\; a = b^k \;\Rightarrow\; T(N) = O(N^k \log N) = O(N \log N)$$

**Búsqueda binaria.** Se divide en 2 mitades pero solo se llama a una (a = 1) de tamaño N/2 (b = 2), y fuera de la llamada recursiva solo compara con el elemento del medio, un trabajo constante, $O(1) = O(N^0)$, así que k = 0.

Como $b^k = 2^0 = 1$ es igual a a, se aplica otra vez el caso $a = b^k$:

$$a = 1, \quad b^k = 2^0 = 1 \Rightarrow a = b^k \Rightarrow T(N) = O(N^k \log N) = O(N^0 \log N) = O(\log N)$$

- **Ordenar por fecha o por prioridad:** copiar la lista es O(N) y ordenarla O(N log N), en total O(N log N).
- **Agregar un pedido:** hallar la posición recorre hasta N elementos y insertar desplaza hasta N, en total O(N). Cargar N pedidos de a uno es $O(N^2)$ en el peor caso. Se acepta porque la consulta por id, que es la operación que se repite, es O(log N).

## 1.6 Casos de prueba

PruebasPedidos ejecuta 44 verificaciones en 15 grupos:

- **Empates:** si dos pedidos tienen la misma fecha o la misma prioridad, el orden lo decide el desempate; y si todo es igual, va primero el id menor.
- **Datos ya ordenados o al revés:** los ordenamientos funcionan aunque los pedidos ya vengan en orden o en orden inverso.
- **Buscar por id:** se encuentran el primero, el del medio y el último. Un id que no existe (uno que cae entre dos existentes, menor que el mínimo, mayor que el máximo o negativo) devuelve que no hay pedido.
- **Lista vacía o con un solo pedido:** ordenar y buscar funcionan igual.
- **Id repetido:** al agregar un pedido con un id que ya existe se lanza una excepción y la lista queda como estaba.
- **Datos originales:** ordenar no modifica el arreglo de entrada ni la lista del gestor; el resultado es una copia.
- **Agregar después de buscar:** al insertar un pedido nuevo, la lista sigue ordenada por id.
- **Un orden no altera el otro:** pedir el orden por fecha no cambia el orden por prioridad, y al revés.
- **Comparador:** un criterio inválido lanza excepción, y los desempates se prueban comparando dos pedidos directamente.
- **Volumen (2000 pedidos):** con ids mezclados y muchas fechas y prioridades repetidas, los dos órdenes quedan correctos y se encuentra cada id.

# 2. Funcionalidad 2: cálculo de la ruta más rápida

## 2.1 Análisis del problema

La organización dispone de una red vial formada por depósitos, centros comunitarios y conexiones dirigidas. Cada conexión habilitada posee un tiempo de viaje fijo y positivo, expresado en minutos. Dados un origen y un destino, se busca el recorrido cuya suma de tiempos sea mínima, no necesariamente el que tenga menos conexiones.
El sistema debe mostrar las ubicaciones atravesadas y el tiempo total. También debe permitir deshabilitar una conexión sin modificar automáticamente el sentido inverso, recalcular con la red actualizada y avisar cuando el destino no sea alcanzable. Los identificadores de las ubicaciones son únicos, y no se admiten tiempos cero o negativos para conexiones habilitadas.
La red se representa con una matriz de adyacencia: tiempos [i] [j] almacena los minutos para ir de la ubicación i a la j y 0 significa que no hay conexión habilitada. Los identificadores externos se guardan en una lista de ubicaciones, que permite relacionarlos con las posiciones de la matriz. La matriz refleja la dirección de circulación: tiempos [i] [j] y tiempos[j] [i] son independientes.

## 2.2 Estrategia elegida: Greedy (algoritmo de Dijkstra)

Se utilizó la estrategia Greedy con el algoritmo de Dijkstra: en cada iteración elige entre las ubicaciones no visitadas aquella con menor tiempo acumulado conocido desde el origen, y confirma ese tiempo como definitivo. Esto es correcto porque todos los tiempos de las conexiones son positivos.

- **Candidatos:** las ubicaciones todavía no visitadas que tienen un tiempo conocido desde el origen.
- **Selección:** escoger la ubicación pendiente con el menor tiempo acumulado (en un empate, se elige el identificador menor para obtener un resultado reproducible).
- **Factibilidad y actualización:** revisar las conexiones habilitadas que salen de la ubicación elegida. Si pasar por ella mejora el tiempo de una ubicación vecina, se actualizan su distancia y su predecesor.
- **Solución:** terminar cuando el destino queda confirmado, o cuando no existen más ubicaciones alcanzables. Se reconstruye el recorrido desde el destino siguiendo los predecesores.

Se mantienen tres arreglos principales: distancia [] (tiempo mínimo conocido, inicialmente infinito salvo origen = 0), visitado [] (ubicaciones definitivas) y anterior [] (predecesor que permite reconstruir el camino). La selección y la actualización se realizan con ciclos sobre la matriz; no se utilizan métodos ajenos a las diapositivas ni bibliotecas externas de grafos.
Ejemplo de ejecución. Se consideran las conexiones dirigidas 1→2 (6), 1→3 (16), 1→4 (30), 2→3 (4), 2→4 (14) y 3→4 (3), con origen 1 y destino 4. La ubicación 5 no posee conexiones.

| Paso | Se confirma | Tiempo fijo | Mejoras observadas |
|---|---|---|---|
| 1 | 1 | 0 | 2=6; 3=16; 4=30 |
| 2 | 2 | 6 | 3: 16→10; 4: 30→20 |
| 3 | 3 | 10 | 4: 20→13 |
| 4 | 4 | 13 | Destino confirmado; fin |

El recorrido óptimo es 1→2→3→4 y tarda 6 + 4 + 3 = 13 minutos. El algoritmo registra en texto cada elección confirmada y cada mejora, de modo que se pueden visualizar las decisiones principales, como pide la consigna.
Si se deshabilita la conexión 2→3, se vuelve a ejecutar Dijkstra sobre la matriz modificada. El nuevo recorrido es 1→3→4 y tarda 16 + 3 = 19 minutos. Si se consulta la ubicación aislada 5 desde 1, se informa que no existe una ruta disponible.

## 2.3 Alternativas descartadas

- **Elegir siempre la conexión saliente más rápida:** es una decisión local que no garantiza el mejor tiempo completo hasta el destino; puede llevar a un camino globalmente más lento..
- **Floyd:** también fue visto en la materia y calcula caminos mínimos entre todos los pares. Sería apropiado para muchas consultas sobre la misma red (la funcionalidad opcional A1), pero aquí se consulta un origen y destino y pueden modificarse conexiones, por lo que no hace falta calcular toda la matriz de pares.
- **Enumerar todas las rutas:** examinar las combinaciones posibles resulta mucho más costoso en redes crecientes que el procedimiento Greedy de Dijkstra.

## 2.4 Complejidad temporal

Sea V la cantidad de ubicaciones. Con la matriz de adyacencia, en cada una de hasta V iteraciones se recorren V posiciones para seleccionar la próxima ubicación y hasta V posiciones para revisar sus conexiones. Por lo tanto, el costo temporal del cálculo es $O(V^2)$.Reconstruir el camino cuesta O(V) en el peor caso: primero se guardan las ubicaciones en orden inverso siguiendo los predecesores y después se recorren de atrás hacia adelante para formar la ruta final, sin inserciones al inicio de la lista. Deshabilitar una conexión existente cuesta O(1), ya que solo se modifica una celda de la matriz. Luego, recalcular la ruta vuelve a costar $O(V^2)$. Agregar una ubicación implica ampliar y copiar la matriz, con un costo de $O(V^2)$.

## 2.5 Decisiones de diseño

- **Dirección de las conexiones:** una conexión 1→2 no implica que exista 2→1. Deshabilitar una solo modifica la celda correspondiente.
- **Valores especiales:** el 0 de la matriz significa ausencia de conexión; las distancias todavía desconocidas usan infinito. Para sumar tiempos se emplea long y evitar el desbordamiento de int.
- **Empates:** entre dos ubicaciones pendientes con la misma distancia se confirma primero la de identificador menor. Si un recorrido alternativo iguala un tiempo ya encontrado, se conserva el predecesor previo; ambos recorridos son óptimos.
- **Casos límite y validaciones:** si el origen coincide con el destino, el recorrido contiene solo esa ubicación y tarda 0. Se rechazan la red nula, las ubicaciones inexistentes, las conexiones con tiempos no positivos y los identificadores repetidos.

- **Organización del código:** RedVial administra los datos de la red, CalculadorRutas implementa Dijkstra y ResultadoRuta almacena los resultados. Para mejorar la legibilidad, el método calcular delega tareas en tres métodos auxiliares: elegirUbicacion, actualizarDistancias y reconstruirRecorrido. Main permite demostrar el funcionamiento y PruebasRutas verifica los resultados.

### 2.6 Casos de prueba

- **Ejemplo de demostración:** se calcula el recorrido 1→2→3→4 de 13 minutos.
- **Recalcular después de deshabilitar:** sin 2→3 se obtiene 1→3→4 en 19 minutos; con otra modificación se obtiene 1→2→4 en 20 minutos; al rehabilitar vuelve el óptimo de 13.
- **Sentido de circulación:** una conexión de ida no habilita automáticamente la vuelta, y deshabilitar una dirección no altera la opuesta.
- **Sin ruta y origen igual a destino:** se informa que una ubicación aislada es inalcanzable, y que viajar desde una ubicación hasta ella misma tarda 0.
- **Otros caminos y empates:** se comprueban caminos directos, rutas alternativas y un caso con soluciones de igual duración.
- **Decisiones y datos:** se revisa que la traza contenga confirmaciones y mejoras, y que las listas devueltas no permitan alterar el resultado interno.
- **Validaciones y volumen:** se comprueban entradas inválidas, tiempos grandes y una red de 200 ubicaciones cuyo recorrido más rápido tiene 199 minutos.

# 3. Funcionalidad 3: selección de cargas prioritarias

### 3.1 Análisis del problema

Un vehículo tiene una capacidad máxima de carga (un entero mayor o igual a 0). Cada lote tiene un identificador, un peso entero positivo y una prioridad entera no negativa. Los lotes no se pueden fraccionar y cada uno se carga una sola vez. El objetivo es elegir los lotes que den la mayor prioridad total sin superar la capacidad, y mostrar la selección, el peso total, la prioridad obtenida y la capacidad restante. Es el problema de la mochila con resolución mediante programación dinámica

El sistema rechaza con una excepción los datos que contradicen estas condiciones: peso no positivo, prioridad negativa, capacidad negativa, vehículo o lista nulos, lotes nulos e ids de lote repetidos.

OBS: Si hay varias selecciones con la misma prioridad total, se elige la que usa los lotes que aparecen primero en la lista. Por ejemplo, con capacidad 5 y los lotes L1 (5, 10), L2 (3, 5) y L3 (2, 5), la lista en ese orden elige [L1] y la lista invertida elige [L3, L2]; la prioridad es 10 en ambos casos. Un lote de prioridad 0 nunca se carga, porque no aporta prioridad.

### 3.2 Organización de los datos

La selección usa tres estructuras:

- **Lista de lotes:** se accede por posición, porque la fila i de la matriz corresponde al lote i−1 de la lista. La lista original no se modifica.

- **Matriz de enteros matriz[i][c]:** tiene n+1 filas (una por lote, más la fila 0 de "ningún lote") y C+1 columnas (una por capacidad, de 0 a C). Guarda la mayor prioridad posible para cada par (lotes disponibles, capacidad).
- **Arreglo de booleanos:** al reconstruir la solución marca qué lotes se cargaron, y con él se arma la lista final respetando el orden original de los lotes.

## 3.3 Estrategia elegida: Programación Dinámica

La idea es armar una tabla, matriz[i][c], donde el valor de cada celda representa la mayor prioridad si solo puedo usar los primeros i lotes y tengo una capacidad c. La fila 0 (ningún lote) vale 0 en todas las columnas, y la fila i corresponde al lote i−1 de la lista. La tabla se llena fila por fila, de menor a mayor y cada celda se calcula teniendo en cuenta las celdas de la fila anterior, que ya están resueltas.

Para calcular una celda hay solo dos opciones con el lote i:

- **No cargarlo:** queda lo mismo que sin este lote, es decir el valor de la celda de arriba, matriz[i-1][c].
- **Cargarlo:** se suma la prioridad del lote. El resultado es su prioridad más matriz[i-1][c - peso]. Solo es posible si el lote entra, es decir si la suma de los pesos es menor o igual a c.

Si el lote entra, se elige el máximo de las dos opciones. Si el lote no entra, la única opción es no cargarlo.

Con $w_i$ el peso y $v_i$ la prioridad del lote i, esto se define así:

$$M[i][c] = \begin{cases} M[i-1][c] & \text{si } w_i > c \\ \max(M[i-1][c],\ M[i-1][c-w_i] + v_i) & \text{si } w_i \leq c \end{cases}$$

La prioridad máxima es la última celda, matriz[n][C]. Para saber qué lotes se cargaron se recorre la matriz desde esa celda hacia arriba: si matriz[i][c] es distinto de la celda de arriba (matriz[i-1][c]), el lote i se cargó y la capacidad baja en su peso; si es igual, no se cargó. Por eso, cuando dos selecciones tienen la misma prioridad, gana la que usa los lotes que aparecen primero en la lista.

**Ejemplo.** Lotes (peso, prioridad): L1 (1, 10), L2 (4, 3), L3 (5, 5), L4 (7, 8), con capacidad 8.

| i \ c | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| 0 (ninguno) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 (L1) | 0 | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 10 |
| 2 (L2) | 0 | 10 | 10 | 10 | 10 | 13 | 13 | 13 | 13 |
| 3 (L3) | 0 | 10 | 10 | 10 | 10 | 13 | 15 | 15 | 15 |
| 4 (L4) | 0 | 10 | 10 | 10 | 10 | 13 | 15 | 15 | 18 |

La prioridad máxima es 18. Reconstrucción: en (4, 8) el valor 18 es distinto del 15 de arriba, así que se carga L4 como parte de la solución y la capacidad pasa a 8 − 7 = 1. Se mueve a la celda (3, 1) y se compara con (2, 1) donde el valor 10 es igual al de arriba, así que L3 y L2 no se cargan. En la celda (1, 1) el valor 10 es distinto del 0 de arriba, así que se carga L1 y la capacidad queda en 0. Como solución quedan L1 y L4, peso 8, prioridad 18, capacidad restante 0.

**Por qué Programación Dinámica.** Para decidir qué hacer con un lote alcanza con conocer la mejor carga con los lotes anteriores y la capacidad que queda, que es el mismo problema pero más chico. Esos problemas chicos se repiten: en el ejemplo, la celda (2, 1) se usa para calcular tanto la celda (3, 1) como la (3, 6). Por eso cada uno se resuelve una sola vez y se guarda en la tabla, y se reutiliza cada vez que hace falta.

La estrategia se reconoce en SelectorCargas: llenarMatriz calcula la matriz con el caso base de la fila 0 y la elección entre cargar y no cargar con Math.max, y reconstruir recorre la matriz hacia atrás para encontrar la solución.

## 3.4 Alternativas descartadas

La otra alternativa considerada fue una estrategia Greedy, en la que se ordenan los lotes por la relación prioridad/peso (de mayor a menor) y se cargan en ese orden mientras entren en la capacidad del vehículo.

Se descartó porque no garantiza la solución óptima. Greedy decide mirar un lote a la vez y nunca se arrepiente: una vez que carga un lote, no puede revisar si convenía más dejar ese espacio para otros. Como cada lote se carga entero, el lote con mejor relación en ese momento puede ocupar un espacio que luego deja capacidad sin aprovechar.

Ejemplo: capacidad 10 y los lotes A (peso 6, prioridad 60), B (peso 5, prioridad 45) y C (peso 5, prioridad 45). Las relaciones prioridad/peso son 10, 9 y 9, así que Greedy elige primero A. Después de A quedan 4 de capacidad y ni B ni C entran (pesan 5), por lo que el resultado es prioridad 60 con 4 unidades sin usar. Pero la mejor selección era B y C ya que pesan 5 + 5 = 10 y suman prioridad 90.

La Programación Dinámica no tiene este problema porque, para cada lote y cada capacidad, calcula las dos posibilidades (cargarlo o no cargarlo) y se queda con la mejor, en lugar de comprometerse con una sola decisión.

## 3.5 Complejidad temporal

Siendo n la cantidad de lotes y C la capacidad del vehículo.

- **Llenar la matriz:** llenarMatriz tiene dos for anidados. El de afuera recorre los lotes (i de 1 a n), es decir n vueltas. El de adentro recorre las capacidades (c de 0 a C), es decir C+1 vueltas por cada vuelta del de afuera. Dentro del for interno solo hay una comparación, una suma y un Math.max, que son operaciones constantes que cuestan O(1). Entonces el total es Θ(n·C).
- **Reconstrucción y armado del resultado:** se recorre la matriz una fila por vez, de la última a la primera, y en cada fila se hace una comparación y, a lo sumo, una resta. Son n pasos de tiempo constante, es decir Θ(n). Armar la lista de lotes

elegidos recorre los n lotes una vez, también Θ(n). Complejidad final: Θ(n) + O(n) = Θ(n)

- **Validaciones:** recorren los lotes una vez, comprobando que ninguno sea nulo y que su id no haya aparecido antes (con un HashSet,cada consulta es O(1)). La complejidad final es O(n).
- **Seleccionar :** se suman los tres costos, Θ(n·C) + Θ(n) + O(n). Así que el total es Θ(n·C).

## 3.6 Casos de prueba

PruebasCargas ejecuta 52 verificaciones en 14 grupos:

- **Ejemplo en clase:** con capacidad 8 y los lotes del ejemplo se eligen los lotes 1 y 4, con prioridad 18, peso 8 y capacidad restante 0.
- **Caso donde Greedy no sirve y programación dinámica si:** con capacidad 10 se eligen los lotes 2 y 3, con prioridad 90.
- **Sobra capacidad:** con los mismos lotes y capacidad 9, se elige solo el lote 1 (prioridad 60) y quedan 3 libres.
- **Casos límite:** capacidad 0, lista vacía, un lote más pesado que la capacidad, ningún lote que entre, un lote que entra justo y todos los lotes entrando.
- **Prioridad 0:** un lote sin prioridad nunca se carga.
- **Empates:** si dos selecciones suman la misma prioridad, gana la que usa los lotes que aparecen primero; con la lista invertida cambia la selección pero no la prioridad.
- **Datos originales:** la lista de lotes no se modifica.
- **Datos inválidos:** peso 0 o negativo, prioridad negativa, capacidad negativa, vehículo nulo, lista nula, lote nulo e id repetido lanzan excepción; la capacidad 0 es válida.
- **Volumen (2000 lotes, capacidad 500):** todos pesan 1 y las prioridades 1 a 2000 se reparten al azar; se eligen 500 lotes, todos entre los de prioridad 1501 a 2000, con prioridad total 875250 y capacidad restante 0.

# 4. Funcionalidad A1: consultas de rutas entre todos los centros

## 4.1 Análisis del problema

La organización dispone de una red vial formada por depósitos, centros comunitarios y conexiones dirigidas, cada una con un tiempo de viaje fijo y positivo. En la funcionalidad 2 se resolvía una consulta puntual mediante Dijkstra; ahora se necesita responder numerosas consultas entre distintos pares de ubicaciones sin volver a realizar un cálculo completo para cada una.

El objetivo es preparar la red una sola vez, conservar los tiempos mínimos y poder reconstruir el recorrido elegido. Si no existe un camino dirigido entre dos ubicaciones, se debe informar. Un mismo origen y destino representa un recorrido de tiempo cero.

Se reutiliza RedVial, que ya representa las conexiones habilitadas mediante una matriz de adyacencia. Los valores iguales a cero en esa matriz indican ausencia de conexión, excepto la diagonal de la matriz de resultados, que debe comenzar en cero. Los identificadores pueden ser arbitrarios y se conservan separados de las posiciones de las matrices.

## 4.2 Estrategia elegida: Programación Dinámica (Floyd-Warshall)

Se eligió Floyd-Warshall, la idea consiste en resolver los caminos mínimos entre todos los pares usando soluciones parciales. En cada etapa k se evalúa si conviene ir de i a j directamente por el mejor camino ya conocido o pasar también por la ubicación k como intermedia.

La matriz inicial M0 tiene 0 en su diagonal, el tiempo de la conexión cuando existe y un valor de infinito cuando no existe conexión. Se recorren todos los posibles puntos intermedios k y, para cada par (i, j), se aplica:

$$M^{k}[i][j] = \text{mín}( M^{k-1}[i][j], M^{k-1}[i][k] + M^{k-1}[k][j] )$$

En el código se actualiza una sola matriz distancias [][], evitando guardar todas las matrices intermedias. Se utilizan tres ciclos anidados, en el orden k, i, j. Cuando no se puede llegar de i a k o de k a j, no se hace la suma. Cuando pasar por k mejora el costo, también se actualiza siguiente [i][j] con el primer paso necesario para reconstruir posteriormente ese camino.

- **Subestructura óptima:** el camino mínimo de i a j pasando por k combina caminos mínimos de i a k y de k a j.
- **Subproblemas superpuestos:** las mejores distancias parciales se reutilizan entre múltiples pares y etapas; por eso se guardan en una matriz.
- **Solución final:** terminados los ciclos, distancias[i][j] guarda el tiempo mínimo para cualquier par alcanzable y siguiente[i][j] permite recuperar su recorrido.

### Ejemplo de procesamiento

Se toma la misma red de la primera entrega: 1→2 (6), 1→3 (16), 1→4 (30), 2→3 (4), 2→4 (14) y 3→4 (3). La ubicación 5 está aislada. El símbolo ∞ indica que no existe camino.

| Origen → destino | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 6 | 16 | 30 | ∞ |
| 2 | ∞ | 0 | 4 | 14 | ∞ |
| 3 | ∞ | ∞ | 0 | 3 | ∞ |
| 4 | ∞ | ∞ | ∞ | 0 | ∞ |

| | | | | | |
|---|---|---|---|---|---|
| 5 | ∞ | ∞ | ∞ | ∞ | 0 |

Matriz inicial M0: todavía no se consideran ubicaciones intermedias.

| **Origen → destino** | **1** | **2** | **3** | **4** | **5** |
|---|---|---|---|---|---|
| 1 | 0 | 6 | 10 | 13 | ∞ |
| 2 | ∞ | 0 | 4 | 7 | ∞ |
| 3 | ∞ | ∞ | 0 | 3 | ∞ |
| 4 | ∞ | ∞ | ∞ | 0 | ∞ |
| 5 | ∞ | ∞ | ∞ | ∞ | 0 |

Matriz final: al incorporar el nodo 2 se mejora 1→3 a 6+4=10 y 1→4 a 6+14=20. Al incorporar el nodo 3 se mejora 1→4 a 10+3=13 y 2→4 a 4+3=7.

Así, la consulta 1→4 devuelve el recorrido 1→2→3→4, con tiempo total 13 minutos; 2→4 devuelve 2→3→4, con 7 minutos. La consulta 1→5 informa que no hay ruta, y 5→5 devuelve 0 minutos y el recorrido [5]. Las consultas se resuelven a partir de las matrices ya preparadas, sin repetir los tres ciclos de Floyd-Warshall.

## 4.3 Alternativas descartadas

**Repetir Dijkstra por cada consulta:** resuelve correctamente un origen y un destino, pero con matriz cuesta O(V²) en cada ejecución. Para numerosas consultas conviene un cálculo inicial común a todos los pares.

**Revisar solamente las conexiones directas:** no sirve porque un camino con ubicaciones intermedias puede ser más rápido; por ejemplo, de 1 a 4 el tiempo directo es 30, pero el mínimo es 13.

**Enumerar todas las rutas (Backtracking):** explora distintas secuencias de ubicaciones para hallar la menor. En redes crecientes puede generar una cantidad muy alta de alternativas, cuando Floyd resuelve el problema en tiempo polinómico.

Floyd-Warshall tiene un costo inicial superior al de una sola ejecución de Dijkstra. La elección se justifica específicamente por el requisito de consultar repetidamente diferentes pares después de un único procesamiento.

## 4.4 Complejidad temporal

Sea V la cantidad de ubicaciones. Inicializar las matrices requiere O(V²). Floyd-Warshall tiene tres ciclos anidados sobre las V ubicaciones (k, i, j) y cada iteración realiza un número constante de operaciones, por lo que el procesamiento inicial cuesta O(V³).

Una vez preparado el resultado, leer la celda distancias [i][j] cuesta O(1). En nuestra implementación los identificadores externos se buscan en una lista, por lo que localizar las posiciones i y j cuesta O(V) en el peor caso. La consulta completa del tiempo, incluyendo esa búsqueda, es O(V). Reconstruir un recorrido sigue como máximo V ubicaciones, de modo que cuesta O(V).

Si se agrega o deshabilita una conexión, se debe crear nuevamente el objeto FloydWarshall sobre la red actualizada, con otro costo de O(V³). No es necesario volver a procesar la matriz entre consultas mientras la red no cambie.

## 4.5 Decisiones de diseño e implementación

**Reutilización:** RedVial conserva la red dirigida existente; FloydWarshall lee esa matriz sin modificarla y ResultadoRuta se reutiliza para mostrar el tiempo y las ubicaciones.

**Matrices:** distancias [][] guarda tiempos mínimos de tipo long, con Long.MAX_VALUE como infinito; siguiente [][] guarda el índice de la próxima ubicación o -1 si no hay ruta.

**Reconstrucción:** se inicia en el origen y se sigue siguiente [actual][destino] con un while hasta llegar al destino. No se hace una nueva búsqueda de rutas.

**Validez de la red:** los tiempos de las conexiones habilitadas son positivos, como exige RedVial. La conexión i→j no implica la inversa j→i.

**Consultas:** consultarTiempo devuelve el tiempo (o -1 si no hay camino); consultarRecorrido devuelve la lista de ubicaciones; consultar devuelve ambos datos reunidos en ResultadoRuta.

**Cambios en la red:** cada instancia de FloydWarshall representa el estado de la red al momento de procesarla. Para consultar una red modificada, se construye una nueva instancia; no se entrega accidentalmente un resultado antiguo como si estuviera actualizado.

Se mantiene la implementación deliberadamente simple: matrices, listas, ciclos for y while. La estrategia de Programación Dinámica está visible en la actualización de distancias mediante el nodo intermedio k, tal como exige la consigna general.

## 4.6 Casos de prueba

PruebasFloydWarshall ejecuta 5.494 verificaciones en 15 grupos. Las pruebas no requieren JUnit y siguen la organización usada en las funcionalidades anteriores:

**Caso de la primera entrega y múltiples consultas:** verifica rutas 1→4 (13), 1→3 (10), 2→4 (7), reconstrucción de trayectos y consultas repetidas sobre el mismo resultado procesado.

**Cambios de red:** al deshabilitar 2→3 y preparar de nuevo, 1→4 pasa a 19 minutos; otras modificaciones y la rehabilitación de una conexión cambian los resultados correctamente.

**Dirección y casos sin ruta:** se comprueba que la ida no habilita la vuelta, que una ubicación aislada es inalcanzable y que origen igual a destino cuesta cero.

**Casos adicionales:** rutas directas e indirectas, ciclos, empates, identificadores no consecutivos, tiempos grandes, datos inválidos y listas de resultados que no alteran el objeto interno.

**Comparación con Dijkstra:** en una red dirigida reproducible de 30 ubicaciones se comparan los tiempos de Floyd y Dijkstra para los 900 pares origen–destino; también se verifica que el recorrido reconstruido utilice conexiones reales y sume el tiempo informado.

**Volumen:** una red de 180 ubicaciones permite validar la matriz de todos los pares, la reconstrucción de un recorrido de 180 ubicaciones y la inexistencia de un camino inverso.

# 5. Funcionalidad B2: distribución de cargas entre vehículos

## 5.1 Análisis del problema

Se tiene un conjunto de lotes y vehículos. Cada lote tiene un identificador, un peso entero positivo y una lista de los vehículos habilitados para transportarlo. Cada vehículo tiene un identificador y una capacidad máxima (un entero mayor o igual a 0). El sistema debe asignar todos los lotes a algún vehículo, sin fraccionarlos ni repetirlos, de modo que cumpla las siguientes restricciones:

- cada lote vaya a un único vehículo, que sea uno de los habilitados para ese lote
- la suma de los pesos de los lotes asignados a un vehículo no supere su capacidad máxima.

Se debe mostrar una distribución válida (qué lotes lleva cada vehículo) o informar que no existe ninguna. No pide minimizar ni maximizar nada, alcanza con encontrar cualquier distribución válida.

Se diferencia de la funcionalidad 3 donde había un solo vehículo y se elegía qué lotes llevar. Acá hay que llevar todos los lotes y decidir en qué vehículo va cada uno.

Reglas y validaciones:

- Una lista de vehículos habilitados vacía significa que el lote no puede ir en ningún vehículo; si existe un lote así, la distribución no existe.
- Se lanza una excepción si la lista de vehículos o la de lotes es nula, si alguna contiene elementos nulos, o si hay ids repetidos (de vehículos o de lotes).
- No se pueden crear lotes con peso no positivo, prioridad negativa o lista de habilitados nula o con elementos nulos, ni vehículos con capacidad negativa.

OBS: La prioridad del lote no interviene en esta funcionalidad. Si hay varias distribuciones válidas, se devuelve la primera que encuentra el algoritmo.

## 5.2 Organización de los datos

La distribución usa cuatro estructuras:

**Lote con vehículos habilitados:** cada Lote guarda los ids de sus vehículos habilitados en un HashSet. puedeIrEn(vehiculo) consulta si el id del vehículo está en ese conjunto. Costo: O(1).
**Arreglo asignacion:** tiene una posición por lote; asignacion[i] es la posición en la lista de vehículos, del vehículo que lleva el lote i. Es la solución que el backtracking va completando.
**Arreglo capacidadRestante:** tiene una posición por vehículo y guarda cuánto lugar le queda al vehiculo en la posicion i. Empieza con la capacidad máxima, se resta al poner un lote y se devuelve al sacarlo. Así, saber si un lote entra cuesta O(1).
**DistribucionCargas:** guarda el resultado (si existe distribución y el arreglo asignacion) junto con copias de las listas de vehículos y de lotes. Con eso reconstruye qué lotes lleva cada vehículo y arma el texto final.

## 5.3 Estrategia elegida: Backtracking

Los lotes se ubican de a uno, en el orden de la lista. Para cada lote se prueban los vehículos en el orden de la lista, con la regla **elegir, explorar, deshacer**:

**Elegir:** si el vehículo está habilitado para el lote y el peso entra en su capacidad restante, se asigna el lote a ese vehículo y se le resta el peso a esa capacidad.
**Explorar:** se intenta ubicar el resto de los lotes del mismo modo. Si se ubican todos, hay distribución y la búsqueda termina.
**Deshacer:** si más adelante algún lote no tiene solucion con esa distribucion, se saca el lote (se devuelve el peso a la capacidad restante) y se prueba con el siguiente vehículo. Si asi y todo ningún vehículo sirve, se vuelve al lote anterior para probar otra opción.

Los vehículos que no están habilitados o donde el peso no entra se descartan sin explorar lo que vendría después, esa seria la poda en este algoritmo de backtracking. La búsqueda se corta apenas se completa una distribución, porque la consigna pide solo una. Si se agotan todas las opciones sin completarla, se informa que no existe.

**Ejemplo.** Vehículos V1 y V2, ambos de capacidad 10. Lotes (peso, vehículos habilitados): L1 (5, V1 y V2), L2 (6, solo V1), L3 (4, V1 y V2) y L4 (5, solo V2). La capacidad restante se muestra como [V1, V2].

| Paso | Intento | Resultado | Capacidad restante |
|---|---|---|---|
| 1 | L1 en V1 | Habilitado y entra | [5, 10] |
| 2 | L2 en V1 | Habilitado, pero 6 > 5: no entra | [5, 10] |
| 3 | L2 en V2 | No está habilitado. L2 no tiene dónde ir: se vuelve atrás | [5, 10] |
| 4 | Se saca L1 de V1 y se prueba L1 en V2 | Habilitado y entra | [10, 5] |
| 5 | L2 en V1 | Habilitado y entra | [4, 5] |
| 6 | L3 en V1 | Habilitado y entra (4 ≤ 4) | [0, 5] |
| 7 | L4 en V1 | No está habilitado | [0, 5] |
| 8 | L4 en V2 | Habilitado y entra (5 ≤ 5) | [0, 0] |
| 9 | No quedan lotes | Todos ubicados: solución | [0, 0] |

La solución es V1 con L2 y L3 (peso 10 de 10) y V2 con L1 y L4 (peso 10 de 10).

La estrategia se reconoce en el código: DistribuidorCargas.asignar es recursivo y, dentro de un for sobre los vehículos, elige (asignacion[i] = v y resta de capacidadRestante), explora (la llamada recursiva asignar(i + 1, ...)) y deshace (suma de nuevo a capacidadRestante cuando esa llamada devuelve false). El caso base es cuando i es igual a la cantidad de lotes, osea cuando ya pude asignar a todos los lotes en los vehiculos.

## 5.4 Alternativas descartadas

**Greedy:** decide un lote a la vez y nunca se arrepiente, así que puede informar que no hay solución cuando sí existe. En el ejemplo de 4.3, pondría L1 en V1; L2 pesa 6, solo puede ir en V1 y ya no entra, por lo que diría que no existe distribución. Sin embargo existe: V1 con L2 y L3, V2 con L1 y L4.
**Fuerza bruta:** siempre explora las V^L combinaciones. Backtracking tiene el mismo peor caso, pero descarta una rama apenas un lote no entra o no está habilitado, y se corta en la primera solución. Por lo que en el caso promedio es resulta mucho mejor usar backtracking
**Programación Dinámica:** se podría, pero con varios vehículos no alcanza con una tabla de lotes por capacidad como en la funcionalidad 3. Habría que recordar cuánto lugar queda en cada vehículo, así que cada columna sería una combinación de lugares restantes (V1, V2, ...) y la tabla se multiplica con cada vehículo que se agrega. Resulta mucho más pesada y difícil que el backtracking, que además se detiene en la primera solución.

## 5.5 Complejidad temporal

Sea L la cantidad de lotes y V la cantidad de vehículos. Casi todo el tiempo se va en el algoritmo *asignar*, que es recursivo, así que se calcula con el teorema maestro.

**a:** la cantidad de llamadas recursivas que hace cada llamada. *asignar* recorre los V vehículos y, por cada uno donde el lote puede ir, vuelve a llamarse para ubicar el lote siguiente. En el peor caso el lote puede ir en todos los vehículos, así que hace V llamadas. Por lo tanto, **a = V**.

**b:** cuánto se reduce el problema en cada llamada recursiva. Cada llamada ubica un lote, así que la llamada siguiente tiene un lote menos por ubicar: pasa de L a L − 1. El problema se achica de a uno, por lo tanto **b = 1**.

**k:** cuánto trabajo hace cada llamada por su cuenta, sin contar las llamadas recursivas. Ese trabajo se escribe como $O(L^k)$. En asignar se realizan todas operaciones constantes fuera de las llamadas recursivas. $O(1) = O(L^0)$, por lo tanto **k = 0**.

$$a = V,\ b = 1,\ k = 0$$

Con estos tres valores se aplica la regla del teorema: si a > 1, el tiempo es Θ(a^(L div b)). Como hay al menos 2 vehículos, a = V > 1, y reemplazando queda:

$$\Theta\left(a^{L \operatorname{div} b}\right) = \Theta\left(V^{L \operatorname{div} 1}\right) = \Theta\left(V^{L}\right)$$

Entonces el costo total del backtracking en el peor caso es **O(V^L)**.

La poda y el corte en la primera solución reducen mucho el trabajo en la práctica, pero no el peor caso, si no existe distribución y el problema aparece recién en el último lote, el algoritmo ya revisó casi todas las combinaciones.

Otras operaciones:

- **Preparación:** crear asignacion y cargar capacidadRestante es O(L + V).
- **Validaciones:** se recorren vehículos y lotes una vez, con un HashSet para detectar ids repetidos (cada consulta es O(1)). Costo: O(V + L).

- **puedeIrEn**: consulta si un id de un vehiculo esta habilitado dentro de un lote. Al realizar una consulta en un hashSet este es constante. O(1)
- **DistribucionCargas:** getLotesDe y getPesoDe recorren los L lotes, O(L). Armar el texto llama a ambos por cada vehículo, O(V · L).

El costo total está dominado por el backtracking por lo tanto es O(V^L) en el peor caso.

## 5.6 Decisiones de diseño

**Lote sin vehículos habilitados:** Para B2 se usa el constructor de cuatro datos ya que importa los ids de los vehiculos habilitados mientras que el constructor anterior era util solo para la funcionalidad 3 que no lo necesitaba. Se puede utilizar uno solo pero requeriria modificar parte de la funcionalidad 3.
**Sin poda previa por peso total:** no se verifica antes de empezar si el peso total supera la capacidad total. Backtracking lo detecta solo, aunque en los casos imposibles tarda más. Se prefirió un algoritmo con menos pasos para explicar.
**Primera solución y casos límite:** se corta al encontrar una distribución válida; cuál se devuelve depende del orden de las listas. Sin lotes la distribución existe y es vacía; con lotes y sin vehículos no existe.
**Peso usado:** getPesoDe sirve para las pruebas y ver que ningún vehículo se pasó de su capacidad.
**Organización del código:** DistribuidorCargas contiene el algoritmo principal (distribuir, asignar y validar), DistribucionCargas almacena el resultado, Lote y Vehiculo son el modelo, Main demuestra el funcionamiento y PruebasDistribucion verifica los resultados.

## 5.7 Casos de prueba

PruebasDistribucion ejecuta 31 verificaciones en 15 grupos:

**Ejemplo de 5.3:** la distribución existe; V1 lleva los lotes 2 y 3 y V2 los lotes 1 y 4. Es el caso donde hay que volver atrás.
**Casos con solución:** un reparto sencillo (lotes de 3, 3 y 6 en dos vehículos de capacidad 6) y uno de capacidad exacta, donde los pesos llenan justo cada vehículo (7 y 5). Sin lotes, la distribución existe y el vehículo no lleva nada.
**Sin solución:** por peso total (lotes por 12 y capacidad total 10); la suma alcanza pero no se puede repartir (tres lotes de 4 y dos vehículos de 6, ya que los lotes no se fraccionan); un lote cuyo único vehículo habilitado no tiene capacidad, un lote sin vehículos habilitados, un lote más pesado que cualquier vehículo y lotes sin ningún vehículo disponible.
**Habilitación por id:** un Vehiculo distinto con el mismo id cuenta como habilitado, y uno con otro id no.
**Datos originales y datos inválidos:** las listas de entrada no se modifican; listas nulas, ids repetidos de lote o de vehículo, lote de peso 0 y lista de habilitados nula lanzan excepción.

**Comparación con fuerza bruta (500 casos al azar):** con entre 1 y 3 vehículos y entre 1 y 6 lotes, con pesos, capacidades y habilitaciones sorteados (mediante una semilla fija). En todos los casos el backtracking y la fuerza bruta coinciden en si existe distribución, y cuando existe, la entregada es válida: cada lote en un solo vehículo habilitado y ningún vehículo por encima

# 6. Funcionalidad B3: rutas alternativas con restricciones

## 6.1 Análisis del problema

Dados un origen, un destino, un tiempo máximo y un conjunto de ubicaciones prohibidas, el sistema debe encontrar todos los recorridos válidos entre origen y destino, y no solo el más rápido como en la funcionalidad 2. Debe mostrar las rutas encontradas con sus tiempos totales.

Un recorrido es válido si cumple las cuatro condiciones a la vez:

- Empieza en el origen y termina en el destino, usando solo conexiones habilitadas y respetando su sentido.
- Su tiempo total, que es la suma de los tiempos de sus conexiones, no supera el tiempo máximo. Una ruta que tarda exactamente el máximo es válida.
- No pasa por ninguna ubicación prohibida.
- No repite ubicaciones.

Se reutiliza RedVial de la funcionalidad 2 sin modificarla: un grafo donde un 0 significa que no hay conexión habilitada y los tiempos válidos son positivos.

Reglas y validaciones:

Si el origen o el destino están prohibidos, ninguna ruta es válida y se informa que no hay rutas, sin lanzar excepción.
Si el origen coincide con el destino, el único recorrido es esa ubicación con tiempo 0, igual que en la funcionalidad 2.
Si ninguna ruta cumple las condiciones, se informa "No existe ninguna ruta que cumpla las condiciones".
Se lanza una excepción si la red es nula, si el tiempo máximo es negativo o si el origen, el destino o alguna ubicación prohibida no existen en la red. Una lista de prohibidas nula equivale a no prohibir ninguna.

OBS: además de mostrar todas las rutas, se indica la de menor tiempo total. Si dos rutas empatan en el menor tiempo, se informa la que el algoritmo encontró primero. Las rutas se muestran en el orden en que se encuentran, que depende del orden de la lista de ubicaciones de la red, y no ordenadas por tiempo.

## 6.2 Organización de los datos

La búsqueda usa estas estructuras. Donde V es la cantidad de ubicaciones y P la cantidad de ubicaciones prohibidas.

**RedVial:** guarda la red con una matriz de adyacencia (tiempos[i][j], con 0 si no hay conexión) y una lista con los ids de las ubicaciones. BuscadorRutas no usa la matriz directamente: trabaja siempre con ids y consulta la red con getUbicaciones() y getTiempoConexion(origen, destino). RedVial traduce cada id a su posición por dentro, buscando el id en su lista, por eso consultar un tiempo cuesta O(V).

**Lista ubicaciones:** los ids de todas las ubicaciones de la red. Sirve para validar los datos y para recorrer los posibles vecinos de cada ubicación.

**Lista caminoActual:** los ids de la ruta que se está armando. Arranca con el origen; se agrega una ubicación al elegirla y se la saca al deshacer. También indica qué ubicaciones ya se usaron, con contains, O(V).

**Lista prohibidas:** los ids prohibidos, tal como los recibe el método. Saber si una ubicación está prohibida cuesta O(P).

**Listas rutas y tiempos:** se guardan las soluciones. rutas tiene una copia de cada ruta válida (una lista de ids) y tiempos tiene el tiempo total de cada ruta en la misma posición i.

**RutasAlternativas:** guarda el resultado (copias de las rutas y de los tiempos), calcula cuál es la de menor tiempo y arma el texto final. Sus métodos devuelven copias, de modo que quien los use no puede modificar el resultado.

## 6.3 Estrategia elegida: Backtracking

La ruta se va armando de a una ubicación: se parte del origen y en cada paso se agrega una ubicación a la que se pueda ir desde la última. Para encontrar todas las rutas se usa la regla elegir, explorar, deshacer:

**Elegir:** desde la última ubicación de la ruta actual (camino actual) se prueba cada conexión hacia otra ubicación. Antes de elegir se comprueba que la conexión esté habilitada, que la ubicación no esté ya en la ruta, que no esté prohibida y que, al sumar el tiempo de la conexión, no se supere el tiempo máximo. Si se cumple todo, se agrega a la ruta actual.

**Explorar:** se repite el proceso desde la ubicación elegida. Si es el destino, la ruta está completa y se guarda una copia con su tiempo.

**Deshacer:** cuando termina la exploración de una ubicación, se la saca de la ruta y se prueba la siguiente ubicación. Así otras rutas pueden volver a pasar por ella.

Las condiciones que se revisan antes de elegir son la poda: si no se cumplen, esa rama no se explora, porque ya se sabe que no puede dar una ruta válida. Como los tiempos son positivos, una ruta que ya superó el máximo no vuelve a ser válida por seguir avanzando. Importante aclarar que, a diferencia de B2, la búsqueda no se corta en la primera solución.

**Ejemplo.** Red de la funcionalidad 2 con la conexión 3→2 (2 minutos) agregada para formar un ciclo: 1→2 (6), 1→3 (16), 1→4 (30), 2→3 (4), 2→4 (14), 3→4 (3) y 3→2 (2). La ubicación 5 no tiene conexiones. Se desea encontrar todas las rutas partiendo del Origen 1 al destino destino 4 con tiempo máximo 25. Se omiten los intentos hacia ubicaciones sin conexión.

| Paso | Intento | Resultado | Ruta actual y su tiempo |
|---|---|---|---|
| 1 | 1→2 | Habilitada y 6 ≤ 25: se elige | [1, 2] · 6 |
| 2 | 2→3 | Habilitada y 10 ≤ 25: se elige | [1, 2, 3] · 10 |
| 3 | 3→2 | El 2 ya está en la ruta: se descarta | [1, 2, 3] · 10 |

| | | | |
|---|---|---|---|
| 4 | 3→4 | Llega al destino con 13: **ruta válida 1** | [1, 2, 3, 4] · 13 |
| 5 | Se saca 4 y luego 3 | Se vuelve al 2 | [1, 2] · 6 |
| 6 | 2→4 | 6 + 14 = 20 ≤ 25, llega al destino: **ruta válida 2** | [1, 2, 4] · 20 |
| 7 | Se saca 4 y luego 2 | Se vuelve al 1 | [1] · 0 |
| 8 | 1→3 | Habilitada y 16 ≤ 25: se elige | [1, 3] · 16 |
| 9 | 3→2 | 16 + 2 = 18 ≤ 25: se elige | [1, 3, 2] · 18 |
| 10 | 2→3 | El 3 ya está en la ruta: se descarta | [1, 3, 2] · 18 |
| 11 | 2→4 | 18 + 14 = 32 > 25: se descarta por el tiempo máximo | [1, 3, 2] · 18 |
| 12 | Se saca 2 | Se vuelve al 3 | [1, 3] · 16 |
| 13 | 3→4 | 16 + 3 = 19 ≤ 25, llega al destino: **ruta válida 3** | [1, 3, 4] · 19 |
| 14 | Se saca 4 y luego 3 | Se vuelve al 1 | [1] · 0 |
| 15 | 1→4 | 30 > 25: se descarta por el tiempo máximo | [1] · 0 |
| 16 | No quedan opciones | Fin de la búsqueda | |

Las rutas válidas son 1→2→3→4 (13 minutos), 1→2→4 (20) y 1→3→4 (19), y la de menor tiempo es 1→2→3→4.

## 6.4 Alternativas descartadas

**Greedy (Dijkstra):** confirma en cada paso la ubicación de menor tiempo y devuelve una sola ruta, la más rápida. B3 pide todas las rutas que cumplen las condiciones, incluidas las más lentas, así que Dijkstra no alcanza.

**Programación Dinámica (Floyd):** calcula el menor tiempo entre todos los pares de ubicaciones y guarda un solo valor por par. No enumera alternativas, y las ubicaciones prohibidas y el tiempo máximo cambian en cada consulta, por lo que no se podría reutilizar una tabla calculada de antemano.

**Backtracking con poda por mejor solución:** cortar las ramas que ya no pueden mejorar la mejor ruta encontrada. Sirve para buscar solo la mejor, pero en B3 una ruta más lenta que la mejor también debe mostrarse, por lo que no se puede descartar por ese criterio.

## 6.5 Complejidad temporal

Sea V la cantidad de ubicaciones y P la cantidad de ubicaciones prohibidas. Casi todo el tiempo se va en backtracking, que es recursivo, así que se analiza con la recurrencia. El peor caso es una red completa (todas las ubicaciones conectadas entre sí), sin ubicaciones prohibidas y con un tiempo máximo que no descarta nada.

- **a = V:** el for recorre las V ubicaciones y por cada una que cumple las condiciones hace una llamada recursiva. Desde cada llamada salen hasta V llamadas.
- **b = 1:** cada llamada recursiva agrega una ubicación a la ruta, así que el problema se achica en 1.
- **k = 2:** lo que hace cada llamada sin contar las recursivas es el for: V vueltas y, en cada una, búsquedas en listas de hasta V elementos (getTiempoConexion, caminoActual.contains, prohibidas.contains). Son V · V = V² pasos.

**Resultado.** Como a > 1, T(V) = O(a^(V div b)) = O(V^V), y el valor de k no cambia este resultado. Esa cota supone que cada llamada hace V llamadas hijas, y eso no ocurre: el for prueba las V ubicaciones, pero descarta las que ya están en la ruta. La primera llamada, con solo el origen en la ruta, hace como mucho V − 1 llamadas; la siguiente, V − 2; y así hasta 1.

Entonces las llamadas que realmente se hacen son V · (V − 1) · (V − 2) · … · 1 = V! y no V · V · … · V = V^V. Por lo tanto, la búsqueda es **O(V!)** en el peor caso.

Las podas (conexión inexistente, ubicación repetida, prohibida o tiempo máximo superado) reducen el trabajo en la práctica, pero no el peor caso.

Otras operaciones:

- **Validaciones:** exigirExistente busca un id en la lista de ubicaciones, O(V). Se aplica al origen, al destino y a cada prohibida: O(V + P · V), que en el peor caso (P = V) es O(V²).
- **RutasAlternativas:** guarda y copia las R rutas, O(R · V); encontrar la de menor tiempo recorre los tiempos, O(R); armar el texto, O(R · V). Todo queda incluido en el costo de la búsqueda.

El costo total está dominado por el backtracking, por lo tanto es O(V!) en el peor caso.

## 6.6 Decisiones de diseño

**Trabajar con ids:** RedVial guarda los tiempos en una matriz que se lee por posición, y CalculadorRutas trabaja con esas posiciones. BuscadorRutas trabaja solo con ids y consulta la red con sus métodos públicos. El costo es que cada consulta busca ids en listas, un O(V) que no cambia el orden factorial del peor caso y que en redes pequeñas no se nota.

**Origen o destino prohibidos:** no hay rutas y se informa, sin lanzar excepción, porque es una consulta válida cuya respuesta es vacía.

**Tiempo máximo:** es inclusivo (una ruta de exactamente ese tiempo es válida). Un valor negativo lanza excepción y con 0 solo es posible la ruta con origen igual a destino.

**Mejor ruta y orden de las rutas:** las rutas se muestran en el orden en que se encuentran, que sigue el orden de la lista de ubicaciones de la red. La de menor tiempo es la primera con ese tiempo.

**Sin comprobación previa de alcanzabilidad:** no se verifica antes de empezar si el destino se puede alcanzar. Si no se puede, la búsqueda explora igual y termina sin rutas. Se prefirió un algoritmo con menos pasos.

**Tipos de dato:** los tiempos de cada conexión son int y el tiempo acumulado y el total de cada ruta son long, para que la suma no se desborde.

**Copias:** al encontrar una ruta se guarda una copia de caminoActual, porque esa lista sigue cambiando durante la búsqueda.

## 6.7 Casos de prueba

PruebasRutasAlternativas ejecuta 65 verificaciones en 21 grupos. Cada verificación imprime OK o FALLO.

**Ejemplo de 6.3:** sin restricciones, de 1 a 4 hay 4 rutas, con tiempos 13, 20, 19 y 30, y la mejor es 1→2→3→4.

**Tiempo máximo:** con máximo 19 quedan 2 rutas; con 13 queda la de exactamente 13 minutos, porque el máximo es inclusivo; con 12 o con 0 no hay rutas.

**Ubicaciones prohibidas:** prohibir la 3 deja 1→2→4 y 1→4; prohibir la 2 y la 3 deja solo 1→4; prohibidas y tiempo máximo juntos; una prohibida que no está en ningún camino no cambia el resultado y una lista nula equivale a no prohibir ninguna.

**Origen o destino prohibido y origen igual a destino:** si el origen o el destino están prohibidos no hay rutas; con origen igual a destino la ruta es esa ubicación con tiempo 0.

**Red con ciclo:** al agregar la conexión 3→2 hay 5 rutas, aparece 1→3→2→4 (32 minutos) y ninguna ruta repite ubicaciones.

**Sin ruta y sentido de las conexiones:** una ubicación aislada es inalcanzable (se informa, el mejor tiempo es −1 y la mejor ruta está vacía); una conexión de ida no habilita la vuelta.

**Conexión deshabilitada:** sin la conexión 2→3 quedan 3 rutas y la mejor es 1→3→4 (19 minutos).

**Red completa:** entre dos ubicaciones de una red completa de 5 hay 16 rutas (1 + 3 + 6 + 6); con tiempo máximo 2 hay 4; sin las ubicaciones 2 y 3 quedan 2.

**Ids distintos de las posiciones:** con ubicaciones agregadas en el orden 30, 20, 10 se encuentran las rutas 10→30 y 10→20→30, y la mejor es la segunda (12 minutos).

**Empates:** con dos rutas de 3 minutos, la mejor es la primera que se encuentra.

**Datos originales y texto:** modificar las listas que devuelve el resultado no lo altera ni modifica la lista de prohibidas; el texto muestra la cantidad de rutas, cada una con su tiempo y la de menor tiempo.

**Datos inválidos:** red nula, origen o destino inexistentes, tiempo máximo negativo y ubicación prohibida inexistente lanzan excepción.

**Comparación con fuerza bruta (500 casos al azar):** redes de 2 a 6 ubicaciones, con ids agregados en orden mezclado para que el id no coincida con la posición, y con conexiones, tiempos, tiempo máximo y ubicaciones prohibidas sorteados mediante una semilla fija. La fuerza bruta arma todas las secuencias de ubicaciones sin repetir y después se queda con las que cumplen las condiciones. En los 500 casos el backtracking encuentra las mismas rutas y el tiempo informado de cada una coincide con la suma de sus conexiones.